TRAVEL AI AGENT

Multi-Agent AI System for Automated Travel Planning

Project Report & Technical Documentation

Field	Details
Project Name	Travel AI Agent
Developer	Abhijeet Singh (abhijeetsingh.ksj@gmail.com)
Framework	CrewAI + LangChain
LLM	Llama-3.3-70B via Groq API (Open Source)
Deployment	Streamlit Community Cloud
Live URL	travel-ai-agent-kqwouktmeoqhcgjupw3i9b.streamlit.app
GitHub	github.com/abhijeetsinghksj/travel-ai-agent
Date	April 2026
Status	✅Deployed & Operational

1. Executive Summary

The Travel AI Agent is a fully automated, multi-agent AI system that plans complete travel itineraries from a single user input. Built using CrewAI and the open-source Llama-3.3-70B language model via Groq API, the system orchestrates four specialized AI agents that work in sequence to deliver a complete travel plan.

Given just a destination and travel dates, the system automatically validates the trip, checks and blocks Google Calendar, generates a detailed day-by-day itinerary with food recommendations, and searches for available flights — all without any additional user input.

Key Achievements

- **100% Open-Source LLM:** Llama-3.3-70B via Groq (free tier, no paid API)
- **Real Google Calendar Integration:** Checks availability and creates blocking events
- **Live Flight Search:** Amadeus API integration (mock data for demo)
- **Cloud Deployed:** Accessible via public Streamlit URL
- **Multi-Agent Architecture:** 4 specialized agents with sequential task execution

2. System Architecture

The system follows a sequential multi-agent pipeline where each agent completes its task and passes context to the next agent. This ensures each agent has full information from previous steps.

2.1 Agent Pipeline Overview

Agent	Role	Tools Used	Output
Agent 1 Input Processor	Validates destination & dates, calculates trip duration, identifies destination type	LLM reasoning only	Trip summary, duration, destination overview
Agent 2 Calendar Manager	Checks Google Calendar for conflicts, blocks dates if free	Google Calendar API v3 (Check + Block)	Calendar status report, event link
Agent 3 Itinerary Planner	Creates day-by-day travel plan with activities, food, tips	LLM reasoning only	Complete N-day itinerary with budget in INR
Agent 4 Flight Specialist	Searches available flights, compares options, recommends best	Amadeus Flight API	Top 3 flights with prices and recommendation

2.2 Data Flow

User Input (Destination + Dates) → Agent 1 (Validate) → Agent 2 (Calendar) → Agent 3 (Itinerary) → Agent 4 (Flights) → Structured Output to UI

2.3 Technology Stack

Component	Technology	Version	Purpose
Agent Framework	CrewAI	0.51.0	Multi-agent orchestration
LLM (Cloud)	Groq API → Llama-3.3-70B	Latest	Language model inference
LLM (Local)	Ollama → Llama3	Optional	Fully local inference
LangChain	langchain + langchain-groq	0.2.16	LLM abstraction layer

Component	Technology	Version	Purpose
Calendar	Google Calendar API v3	2.170.0	Calendar check & block
Flights	Amadeus for Developers	5.1.0	Flight availability search
Frontend	Streamlit	1.45.0	Web UI
Backend	FastAPI	0.115.12	REST API endpoints
Auth	Google OAuth2 / Service Account	2.40.1	Google API authentication

3. Project Structure

File / Folder	Description
agents/travel_crew.py	Core file — defines all 4 CrewAI agents, tasks, and the main run_travel_crew() function
tools/calendar_tool.py	Google Calendar integration — check availability and create blocking events. Supports Service Account (cloud) and OAuth (local)
tools/flight_tool.py	Amadeus flight search — searches real flights, falls back to mock data if API not configured
config/llm_config.py	LLM configuration — auto-detects Groq vs Ollama, returns LangChain LLM object
config/setup_google_auth.py	One-time Google OAuth setup helper for local development
ui/app.py	Streamlit frontend — beautiful dark-themed UI with 4 result tabs
api.py	FastAPI backend — REST endpoints for /plan-trip, /check-calendar, /search-flights
main.py	Entry point — supports CLI, Streamlit UI, FastAPI, and demo modes
requirements.txt	Python dependencies with pinned versions for reproducibility
.env.example	Template for environment variables — copy to .env and fill in keys
config/credentials.json	Google OAuth credentials (NOT committed to git)
config/token.json	Google OAuth token (NOT committed to git)

4. Complete Setup Guide

4.1 Prerequisites

- Python 3.11 or higher
- Git installed
- Groq API key (free at console.groq.com)
- Google Cloud account (for Calendar integration)

4.2 Local Installation

Step 1: Clone and install dependencies

```
git clone https://github.com/abhijeetsinghksj/travel-ai-agent.git
cd travel-ai-agent && pip install -r requirements.txt
```

Step 2: Configure environment

```
copy .env.example .env    # Windows cp .env.example .env      # Mac/Linux
```

Step 3: Set up Google Calendar (one-time)

```
python config/setup_google_auth.py --authenticate
```

Step 4: Run the application

```
python -m streamlit run ui/app.py
```

4.3 Required Environment Variables

Variable	Required	Description	Where to Get
GROQ_API_KEY	YES	Groq API key for LLM inference	console.groq.com/keys
LLM_MODEL	YES	Model name (llama-3.3-70b-versatile)	Set in .env file
GOOGLE_CREDENTIALS_PATH	YES (local)	Path to OAuth credentials JSON	Google Cloud Console

Variable	Required	Description	Where to Get
GOOGLE_SERVICE_ACCOUNT_JSON	YES (cloud)	Service account JSON string	Google Cloud Console
GOOGLE_CALENDAR_ID	YES	Calendar ID (use 'primary')	Google Calendar settings
AMADEUS_CLIENT_ID	Optional	Amadeus API client ID	developers.amadeus.com
AMADEUS_CLIENT_SECRET	Optional	Amadeus API secret	developers.amadeus.com
HOME_CITY	YES	Your home city for flight searches	Set in .env (default: Delhi)

5. API Reference

The FastAPI backend exposes the following REST endpoints. Start the server with: python main.py --api

Method	Endpoint	Description	Example Body
GET	/	Health check and endpoint list	—
POST	/plan-trip	Async trip planning — returns job_id	{"destination":"Goa","start_date":"2026-05-09","end_date":"2026-05-14"}
POST	/plan-trip/sync	Synchronous trip planning (waits for result)	{"destination":"Goa","start_date":"2026-05-09","end_date":"2026-05-14"}
GET	/job/{job_id}	Poll for async job status and result	—
POST	/check-calendar	Check calendar availability only	{"start_date":"2026-05-09","end_date":"2026-05-14"}
POST	/search-flights	Search flights only	{"destination":"Goa","departure_date":"2026-05-09"}
GET	/docs	Interactive Swagger API documentation	—

6. Google Calendar Integration

6.1 Authentication Methods

The system supports two authentication methods depending on the deployment environment:

Method	When Used	Setup Required
Service Account	Cloud deployment (Streamlit Cloud, any server)	Create service account in Google Cloud → Download JSON → Share calendar with service account email → Add JSON to Streamlit secrets
OAuth 2.0	Local development only	Download credentials.json → Run setup_google_auth.py → Browser authorization → token.json saved automatically

6.2 Calendar Operations

- **Check Availability:** Lists all events between start and end dates, filters out transparent events
- **Block Dates:** Creates an all-day event with emoji title, description from itinerary, and email/popup reminders
- **Service Account Email:** travel-agent-service@travel-ai-agent-492806.iam.gserviceaccount.com

7. Cloud Deployment Guide

7.1 Streamlit Community Cloud (Current)

1. Push code to GitHub (github.com/abhijeetsinghksj/travel-ai-agent)
2. Go to share.streamlit.io → New app → Select repo → Set main file: ui/app.py
3. Click Advanced Settings → Paste secrets (GROQ_API_KEY, GOOGLE_SERVICE_ACCOUNT_JSON, etc.)
4. Click Deploy — app live in 2-5 minutes

7.2 Streamlit Secrets Configuration

```
GROQ_API_KEY = "gsk_your_key_here" LLM_MODEL = "llama-3.3-70b-versatile" HOME_CITY = "Delhi"
GOOGLE_CALENDAR_ID = "your@gmail.com" GOOGLE_SERVICE_ACCOUNT_JSON = '''{...json
content...}'''
```

7.3 Live Application

- **URL:** travel-ai-agent-kqwouktmeoqhcgjupw3i9b.streamlit.app
- **GitHub:** github.com/abhijeetsinghksj/travel-ai-agent
- **Cost:** Free (Streamlit Community Cloud free tier)

8. Known Issues & Solutions

Issue	Root Cause	Solution Applied
'tool' import error from crewai	crewai version mismatch between local and cloud	Try/except fallback chain across crewai, crewai.tools, langchain_core.tools
Agent iteration limit reached	CrewAI agents looping without completing	Replaced CrewAI execution with direct LLM calls in run_travel_crew()
Groq model decommissioned	llama3-70b-8192 removed by Groq	Updated to llama-3.3-70b-versatile
Google Calendar 403 error	App not verified by Google	Added test user in OAuth consent screen
Calendar not working on cloud	OAuth requires browser, not available on server	Implemented Service Account authentication
langchain version conflicts	crewai 0.28.8 requires langchain<0.2 but langchain-groq needs >=0.2	Upgraded to crewai 0.51.0 compatible with langchain 0.2.x
Git push rejected	Remote had uncommitted changes from GitHub edits	Used git pull --rebase before push

9. Future Enhancements (Phase 2 & 3)

Phase 2 — Core Improvements (1-2 weeks)

- **Real Flight Booking:** Integrate Amadeus production API with real-time pricing and seat availability
- **Hotel Search Agent:** Add Agent 5 for hotel/accommodation recommendations using Booking.com or Hotels.com API
- **Budget Optimizer:** Add Agent 6 that optimizes the entire trip within a specified budget
- **Weather Agent:** Integrate OpenWeatherMap API to include weather forecasts in the itinerary
- **WhatsApp Notifications:** Send itinerary and calendar confirmation via Twilio WhatsApp API

Phase 3 — Advanced Features (1 month)

- **Memory & Preferences:** Store user preferences (cuisine, travel style, budget) using vector database (ChromaDB)
- **Multi-city Trip Planning:** Support complex itineraries with multiple destinations and connecting flights
- **PDF Export:** Generate downloadable PDF itinerary with maps, photos, and booking links
- **Real-time Collaboration:** Allow multiple users to collaborate on a trip plan simultaneously
- **Voice Input:** Accept voice commands using Whisper API for hands-free trip planning
- **Mobile App:** React Native app with push notifications for trip reminders

Phase 4 — Enterprise Features (3 months)

- **Corporate Travel Policy:** Enforce company travel policies (budget limits, approved airlines)
- **Expense Management:** Automatic expense report generation with receipt scanning
- **Team Travel Coordination:** Plan group trips considering all team members' calendar availability
- **Analytics Dashboard:** Track travel spend, popular destinations, and usage patterns

10. Architecture Diagram (Text)

```
USER INPUT (Destination + Start Date + End Date)
        |
        v
  [AGENT 1: Input Processor]  <-- Llama-3.3-70B (Groq)
  Validates dates, calculates duration, identifies trip type
        |
        v
  [AGENT 2: Calendar Manager]  <-- Google Calendar API v3
  Checks availability --> Blocks dates if free
        |
        v
  [AGENT 3: Itinerary Planner]  <-- Llama-3.3-70B (Groq)
  Creates day-by-day plan with food, activities, budget
        |
        v
  [AGENT 4: Flight Specialist]  <-- Amadeus Flight API
  Searches flights --> Compares --> Recommends best option
        |
        v
  STREAMLIT UI  (4 tabs: Trip Summary | Calendar | Itinerary | Flights)
```

11. Testing & Validation

11.1 Test Cases Verified

Test Case	Input	Expected Output	Status
Basic trip planning	Goa, 2026-05-09 to 2026-05-14	5-day itinerary + flights	✅Pass
Calendar availability check	Any future dates	Free/busy status	✅Pass
Calendar blocking	Delhi trip, May dates	Event created in Google Calendar	✅Pass
Flight search (mock)	Delhi to Goa	3 mock flight options with prices	✅Pass
Invalid date handling	End before start	Error message shown	✅Pass
Cloud deployment	Streamlit Cloud URL	App loads and runs	✅Pass
Service account auth	Cloud environment	Calendar works without browser	✅Pass

11.2 Running Tests Locally

```
# Test individual tools python -m streamlit run ui/app.py # Use the 'Test Individual Tools' section at the bottom of the UI  # Test full crew from command line python main.py --demo
```

12. Quick Reference Card

Action	Command
Run Streamlit UI	python -m streamlit run ui/app.py
Run FastAPI server	python main.py --api (then open localhost:8000/docs)
Run CLI mode	python main.py
Run demo trip	python main.py --demo
Setup Google auth	python config/setup_google_auth.py --authenticate
Push to GitHub	git add . && git commit -m 'message' && git push
Install dependencies	pip install -r requirements.txt
Check Groq models	Visit console.groq.com/docs/models

Service	Free Tier	URL
Groq API (LLM)	14,400 tokens/min, unlimited requests	console.groq.com
Streamlit Cloud	Free hosting, 1GB RAM	share.streamlit.io
Google Calendar API	Free for personal use	console.cloud.google.com
Amadeus API	Test environment free	developers.amadeus.com
GitHub	Free public repos	github.com

— End of Report —

Travel AI Agent | Built by Abhijeet Singh | April 2026